

Chapter: LC 1319 - Number of Operations to Make Network Connected

McKinsey

Salesforce

NVIDIA

Problem Concept & Core Logic

- Basically, we want to remove **redundant edges** and place them somewhere else to connect the network.
- If **n** computers are to be connected, then **(n-1)** edges are required. Otherwise, it is impossible to connect them.

Algorithm Details & Approach

Step-by-step logic using DSU (Disjoint Set Union):

- *"Basically redundant edge hata ke useful jagah connect krne he"*
- An edge is considered redundant if both of its endpoints belong to the same component.
 - If both endpoints belong to the same component → extra++. These extra edges can be used to connect disconnected components.
 - Else → merge them into the same component using standard DSU.
- *"Number of extra edge count kr lenge and number of component count kr lenge"*
- After processing all connections (*"Saare connection krne ke baad ab number of components dekh lenge, jiske liye unique parent count kr lenge"*):
 - Edges needed to connect all components: $\text{need} = \text{components} - 1$

- "Agar extra edges $\geq$ need he, to need return kr denge as minimum hi edge chahiye to connect all"

Implementation Source Code (C++)

```

class dsu{
public:

    vector<int> parent;
    vector<int> size;
    dsu(int n){
        parent.resize(n);
        size.resize(n,1);

        for(int i=0;i<n;i++){
            parent[i]=i;
        }
    }

    int findparent(int node){

        if(parent[node]==node){
            return node;
        }

        parent[node]=findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u,int v){
        int pu=findparent(u);
        int pv=findparent(v);

        if(pu==pv) return;

        if(size[pu]<size[pv]) swap(pu,pv);

        parent[pv]=pu;
        size[pu]+=size[pv];
    }
};

class Solution {
public:
    int makeConnected(int n, vector<vector<int>>& connections) {
        //basically redundant edge hata ke useful jagah connect krne he
        //an edge is redundant if both endpoint to same component
        //number of extra edge count kr lenge
        //number of component count kr lenge
        //need=component-1 edges
    }
};

```

```
//agar edges>=need he to to need return kr denge as minimum hi edge  
chahiye to cpnnect all
```

```
dsu ds(n);  
int extra=0;
```

```
for(auto &it:connections){  
    int u=it[0];  
    int v=it[1];  
  
    if(ds.findparent(u)==ds.findparent(v)){  
        extra++;  
    }  
  
    else{  
        ds.unionit(u,v);  
    }  
}
```

```
//saare connection krne ke baad ab number of components dekh lenge  
//jiske liye unique parent count kr enge
```

```
int components=0;  
int need=0;  
unordered_set<int> st;  
for(int i=0;i<n;i++){  
    st.insert(ds.findparent(i));  
}
```

```
components=st.size();  
need=components-1;
```

```
if(extra>=need){  
    return need;  
}
```

```
else{  
    return -1;  
}
```

```
}
```

```
};
```

